



**ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR**
Membre de
HONORIS UNITED UNIVERSITIES



**École Marocaine des Sciences de l'Ingénieur
EMSI**

Ingénieur Informatique — 4IIR

Rapport du projet

Intitulé :

ÉléganceShop

Application E-Commerce avec Spring Boot

Présenté par :

MARIYAM LAHRAR - NABIL ElhAMDANI

Encadrant : PR. Hssaini Abdelilah

École SUPERIEURE d'Ingénieur

ANNEE UNIVERSITAIRE 2025–2026

Remerciements

Nous adressons nos sincères remerciements à notre encadrant **Hssaini Abdelilah** pour son soutien constant, sa disponibilité et ses précieux conseils tout au long de ce projet. Son expertise en développement logiciel et son engagement ont été des piliers essentiels dans la réalisation de ce travail.

Nos remerciements s'adressent également au corps professoral et administratif de l'École Marocaine des Sciences de l'Ingénieur. Leurs efforts continus et leur soutien tout au long de notre formation ont été déterminants pour notre développement académique et professionnel.

Enfin, nous remercions nos familles et amis pour leur soutien moral et leurs encouragements constants, qui ont constitué une source de motivation tout au long de ce parcours.

Merci à tous !

Résumé

Ce projet de fin d'études porte sur la conception et le développement d'une application web e-commerce intitulée **ÉléganceShop**, développée dans le cadre du standard Jakarta EE avec le framework **Spring Boot**. L'objectif principal est de mettre en œuvre une plateforme de gestion commerciale permettant aux utilisateurs d'explorer un catalogue de produits, de gérer des commandes et un panier d'achat, et aux administrateurs de piloter l'ensemble du système via un tableau de bord dédié.

La plateforme repose sur une architecture **MVC en trois couches** (Présentation, Service, DAO) garantissant un faible couplage et une séparation claire des responsabilités. Le backend est développé avec **Spring Boot**, **Spring MVC**, **Spring Security** et **Spring Data JPA**, tandis que le frontend est réalisé avec **Thymeleaf** et **Bootstrap**. La persistance des données est assurée par une base de données relationnelle **H2** intégrée pour le développement.

Le système intègre un service d'authentification sécurisé, un CRUD complet pour les articles et les commandes, un service de recherche multicritère, un gestionnaire de panier, ainsi que des filtres de sécurité robustes. L'interface administrateur offre un tableau de bord synthétique permettant le suivi en temps réel des clients, articles, commandes et revenus.

Mots-clés : Spring Boot, Spring MVC, Spring Security, Thymeleaf, Bootstrap, Spring Data JPA, H2, E-commerce, Jakarta EE

Abstract

This end-of-studies project focuses on the design and development of an e-commerce web application named **ÉléganceShop**, built within the Jakarta EE standard using the **Spring Boot** framework. The primary objective is to deliver a commercial management platform enabling users to browse a product catalog, manage orders and a shopping cart, while administrators can oversee the entire system through a dedicated dashboard.

The platform is built on a three-tier MVC architecture (Presentation, Service, DAO) ensuring low coupling and clear separation of concerns. The backend is implemented with Spring Boot, Spring MVC, Spring Security, and Spring Data JPA, while the frontend leverages Thymeleaf and Bootstrap. Data persistence is handled by an embedded H2 relational database for development purposes.

The system incorporates a secure authentication service, full CRUD operations for articles and orders, a multi-criteria search service, a cart management module, and robust security filters. The admin interface provides a synthetic dashboard for real-time monitoring of clients, articles, orders, and revenue.

Keywords: Spring Boot, Spring MVC, Spring Security, Thymeleaf, Bootstrap, Spring Data JPA, H2, E-commerce, Jakarta EE

Contents

Remerciements	1
Résumé	2
Abstract	3
Liste des figures	5
Introduction Générale	6
1 Contexte Général du Projet	7
1.1 Présentation Générale	7
1.1.1 Contexte	7
1.2 Problématique du Projet	7
1.3 Objectifs du Projet.....	8
1.4 Analyse des Besoins	8
1.4.1 Identification des Acteurs	8
1.4.2 Besoins Fonctionnels	8
1.4.3 Besoins Non Fonctionnels	9
1.5 Architecture et Patrons de Conception	9
1.5.1 Architecture en Couches	9
1.5.2 Patron de Conception MVC	10
2 Étude Conceptuelle	11
2.1 Diagrammes de Cas d'Utilisation	11
2.1.1 Diagramme Global des Cas d'Utilisation	11
2.1.2 Module Authentification et Profil	11
2.1.3 Module Catalogue et Recherche	11
2.1.4 Module Panier et Commandes	12
2.1.5 Module Administration	12
2.2 Diagrammes de Classes	12
2.2.1 Entités du Domaine	12
2.2.2 Architecture des Classes Backend	12
2.3 Diagrammes de Séquence	13
2.3.1 Authentification d'un Utilisateur	13
2.3.2 Ajout d'un Article au Panier	13
2.3.3 Validation d'une Commande	13
3 Étude Technique et Implémentation	14
3.1 Introduction	14
3.2 Technologies Utilisées	14
3.2.1 Spring Boot	14
3.2.2 Spring MVC	14
3.2.3 Spring Security	14
3.2.4 Spring Data JPA	14

3.2.5	Thymeleaf	15
3.2.6	Bootstrap 5	15
3.2.7	Base de Données H2	15
3.3	Comparaison des Technologies	15
3.4	Analyse des Interfaces Utilisateur	23
3.4.1	Interface Boutique (Côté Client)	23
3.4.2	Tableau de Bord Administrateur	23
3.4.3	Interface de Connexion	23
3.4.4	Gestion des Articles (Admin).....	24
3.4.5	Gestion des utilisateurs	25
3.4.6	Gestion des commandes	26
3.4.7	Gestion du Panier	27
Conclusion Générale.....		28
Bibliographie.....		29

List of Figures

Figure 1.1: Architecture en trois couches de l'application ÉléganceShop

Figure 2.1: Diagramme global des cas d'utilisation de l'application ÉléganceShop

Figure 2.2: Diagramme de classe de l'application ÉléganceShop

Figure 2.3: Diagramme de classes de l'architecture Backend (Controllers–Services–Repositories)

Figure 2.4: Diagramme de séquence global de l'architecture

Figure 2.5: Diagramme de séquence authentification d'un utilisateur

Figure 2.6: Diagramme de séquence ajout au panier

Figure 2.7: Diagramme de séquence validation d'une commande

Table 3.1: Comparaison des frameworks backend Java

Table 3.2: Comparaison des moteurs de templates

Figure 3.3: Page principale de la boutique ÉléganceShop (vue client authentifié)

Figure 3.4: Tableau de bord administrateur (/admin/dashboard)

Figure 3.5: Page de login

Figure 3.6: Page des gestion des articles

Figure 3.7: Page des gestion des utilisateurs

Figure 3.8: Page des gestion des utilisateurs

Figure 3.9: Page de gestion du panier

Introduction Générale

La transformation numérique a profondément modifié les pratiques commerciales, plaçant les applications web e-commerce au cœur des stratégies de distribution modernes. Ces plateformes offrent aux entreprises un accès élargi à leurs clients tout en simplifiant la gestion des catalogues, des commandes et des stocks.

Ce projet s'inscrit dans ce contexte et vise la conception et le développement d'une application web e-commerce dénommée **ÉléganceShop**. Cette application, développée avec le framework **Spring Boot** conformément au standard Jakarta EE, propose une solution complète couvrant la gestion des utilisateurs, des articles, des commandes et d'un panier d'achat, le tout sécurisé par des mécanismes robustes de contrôle d'accès.

Ce rapport est organisé en trois chapitres principaux :

- **Chapitre 1 — Contexte général du projet** : présente la problématique, les objectifs, l'analyse des besoins fonctionnels et non fonctionnels, ainsi que les choix architecturaux retenus.
- **Chapitre 2 — Étude conceptuelle** : expose la modélisation UML du système à travers les diagrammes de cas d'utilisation, de classes et de séquence.
- **Chapitre 3 — Étude technique et implémentation** : détaille les technologies utilisées, la structure du projet Spring Boot, la configuration de la sécurité, et présente les interfaces graphiques réalisées.

Chapitre 1

Contexte Général du Projet

1.1 Présentation Générale

1.1.1 Contexte

L'évolution rapide des technologies web et la généralisation du commerce en ligne ont profondément transformé les modèles économiques traditionnels. Les entreprises, qu'elles soient de grande envergure ou de petite taille, cherchent à numériser leurs processus commerciaux afin d'atteindre une clientèle plus large, de réduire les coûts opérationnels et d'améliorer l'expérience utilisateur.

Dans ce contexte, le développement d'applications web e-commerce robustes, sécurisées et maintenables constitue un enjeu majeur. Le framework **Spring Boot**, s'appuyant sur l'écosystème Spring et le standard Jakarta EE, offre une solution de référence pour la mise en œuvre de telles applications grâce à son approche basée sur la convention plutôt que la configuration, son système d'injection de dépendances et ses modules dédiés à la sécurité, la persistance et la présentation.

Le présent projet vise à concevoir et développer une application web e-commerce complète, dénommée **ÉléganceShop**, intégrant l'ensemble des fonctionnalités attendues d'une plateforme commerciale moderne : gestion des comptes utilisateurs, catalogue de produits, gestion des commandes et panier d'achat, le tout sécurisé par des mécanismes de contrôle d'accès.

1.2 Problématique du Projet

Le développement d'une application e-commerce complète soulève plusieurs problématiques fondamentales :

- La nécessité de concevoir une architecture logicielle maintenable, avec un faible couplage entre les couches Présentation, Service et DAO.
- La mise en place d'un système d'authentification et de contrôle d'accès robuste distinguant les utilisateurs standards des administrateurs.
- La gestion cohérente des articles (création, modification, suppression, recherche multicritère) et des commandes associées.
- L'implémentation d'un panier d'achat permettant l'ajout, la modification et la validation de commandes.
- La réalisation d'une interface utilisateur ergonomique et responsive, adaptée aux différents profils (client et administrateur).

1.3 Objectifs du Projet

L'objectif général de ce projet est de concevoir et développer une application web e-commerce intégrant l'ensemble des services requis par la spécification. Plus précisément, les objectifs visés sont :

- Implémenter un **service d'authentification** sécurisé avec gestion des rôles (USER / ADMIN).
- Développer un **CRUD complet pour les utilisateurs**.
- Développer un **CRUD complet pour les articles**, incluant la gestion des identifiants, descriptions, prix, dates d'expiration et quantités en stock.
- Mettre en place un **service de recherche multicritère** sur le catalogue d'articles.
- Réaliser un **CRUD pour les commandes**, chaque commande devant contenir au moins un article pour un utilisateur donné.
- Implémenter un **service de gestion du panier** permettant l'ajout, la modification et la validation d'une commande.
- Configurer des **filtres de sécurité** (Spring Security) pour protéger les ressources selon le rôle de l'utilisateur.
- Respecter les **principes de faible couplage** entre les couches Présentation, Service et DAO.
- Fournir une **interface d'administration** avec tableau de bord synthétique.

1.4 Analyse des Besoins

1.4.1 Identification des Acteurs

Le système implique deux types d'acteurs principaux :

- **Utilisateur (Client)** : acteur principal pouvant s'inscrire, se connecter, parcourir le catalogue, ajouter des articles au panier et passer des commandes.
- **Administrateur** : acteur disposant de droits étendus permettant la gestion complète des articles, des utilisateurs et des commandes, ainsi que l'accès au tableau de bord.

1.4.2 Besoins Fonctionnels

Gestion des comptes utilisateurs : Inscription, connexion et déconnexion sécurisées. Distinction des rôles USER et ADMIN. Gestion des sessions et des tokens de sécurité.

Gestion du catalogue d'articles : Création d'articles avec identifiant, description, prix, date d'expiration et quantité en stock. Lecture, modification et suppression d'articles. Recherche multicritère (par description, prix, disponibilité en stock, date d'expiration).

Gestion des commandes : Création, consultation, modification et suppression de commandes. Chaque commande doit contenir au moins un article et être associée à un utilisateur. Suivi du statut des commandes (en attente, validée, annulée).

Gestion du panier : Ajout d'articles au panier avec gestion des quantités. Modification et suppression d'articles du panier. Validation du panier pour créer une commande.

Tableau de bord administrateur : Vue synthétique du nombre de clients inscrits, articles disponibles, commandes et revenus validés.

1.4.3 Besoins Non Fonctionnels

Sécurité : Authentification basée sur Spring Security avec hachage des mots de passe (BCrypt). Contrôle d'accès par rôle (ROLE_USER, ROLE_ADMIN). Protection CSRF intégrée.

Performance : Temps de réponse acceptable pour les opérations CRUD et les recherches. Utilisation de Spring Data JPA pour l'optimisation des requêtes.

Maintenabilité : Architecture en couches avec faible couplage. Utilisation de l'injection de dépendances Spring. Code structuré et documenté.

Compatibilité : Interface responsive grâce à Bootstrap 5. Compatible avec les navigateurs web modernes.

Extensibilité : Architecture modulaire permettant l'ajout de nouvelles fonctionnalités sans refonte majeure.

1.5 Architecture et Patrons de Conception

1.5.1 Architecture en Couches

Le projet adopte une **architecture MVC en trois couches** conformément aux principes de faible couplage imposés par la spécification :

Couche Présentation (View) : Réalisée avec Thymeleaf et Bootstrap. Responsable de l'affichage des données et des interactions utilisateur. Ne contient aucune logique métier.

Couche Service (Business Logic) : Contient toute la logique métier de l'application. Réalise les opérations CRUD, les vérifications métier et la coordination entre les contrôleurs et les repositories.

Couche DAO (Data Access Object) : Gérée par Spring Data JPA via des interfaces JpaRepository. Responsable de la persistance et de la récupération des données depuis la base H2.

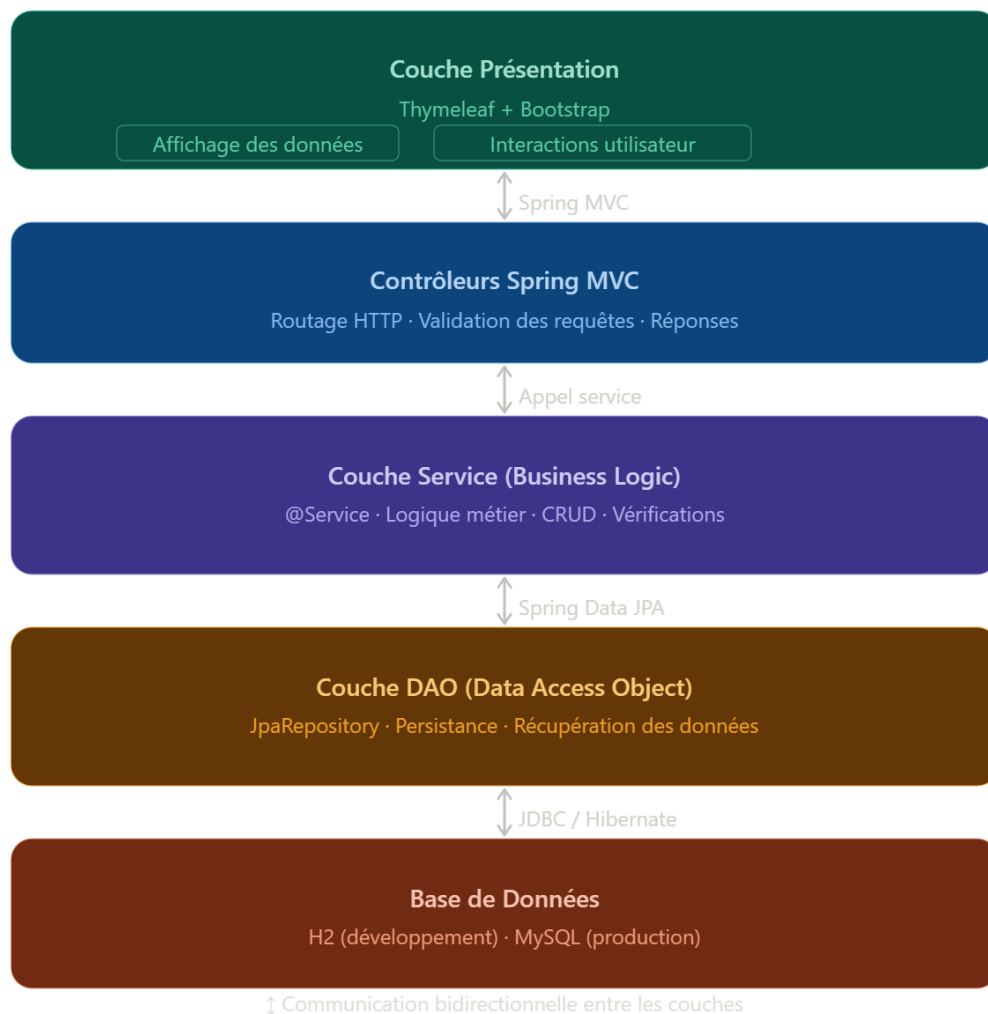


Figure 1.1: Architecture en trois couches de l'application ÉléganceShop

1.5.2 Patron de Conception MVC

Le patron **Model-View-Controller (MVC)** est au cœur de l'organisation de l'application :

- **Model** : Les entités JPA (User, Article, Commande, Panier) représentent les données du domaine, mappées sur les tables de la base de données via Hibernate.
- **View** : Les templates Thymeleaf génèrent dynamiquement les pages HTML. Bootstrap assure le rendu responsive.
- **Controller** : Les contrôleurs Spring MVC (@Controller) reçoivent les requêtes HTTP, délèguent le traitement aux services et retournent la vue appropriée.

Chapitre 2

Étude Conceptuelle

Ce chapitre présente la modélisation UML de l'application ÉléganceShop. L'objectif est de formaliser les interactions entre les acteurs et le système, ainsi que la structure statique et dynamique de l'application avant son implémentation.

2.1 Diagrammes de Cas d'Utilisation

2.1.1 Diagramme Global des Cas d'Utilisation

Le diagramme global présente l'ensemble des fonctionnalités offertes par la plateforme ÉléganceShop selon les deux acteurs principaux : l'Utilisateur (Client) et l'Administrateur.

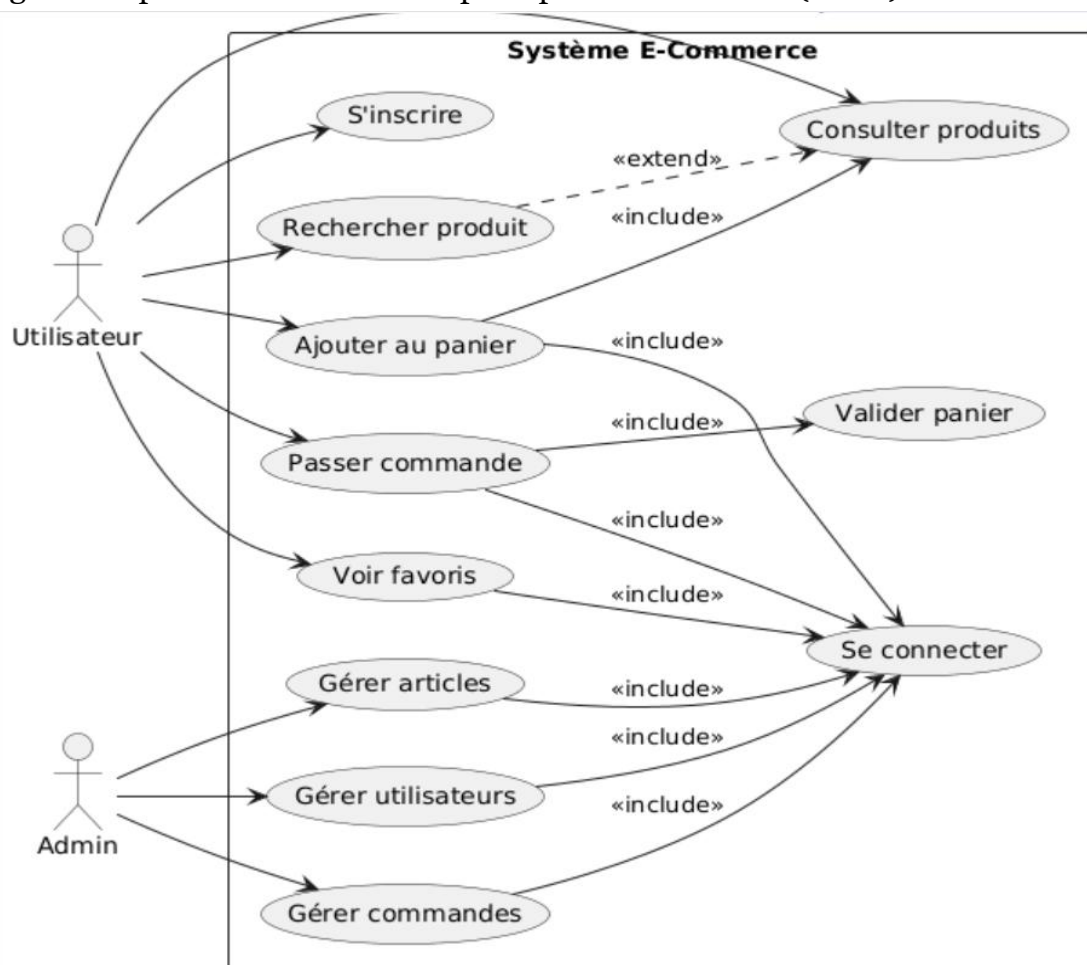
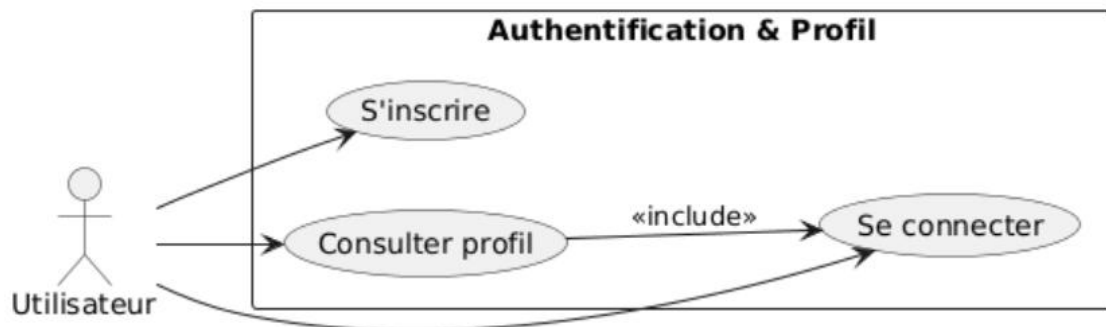


Figure 2.1: Diagramme global des cas d'utilisation de l'application ÉléganceShop

Les fonctionnalités sont regroupées en cinq modules : **Authentification et Profil**, **Catalogue d'Articles**, **Panier**, **Gestion des Commandes**, et **Administration**.

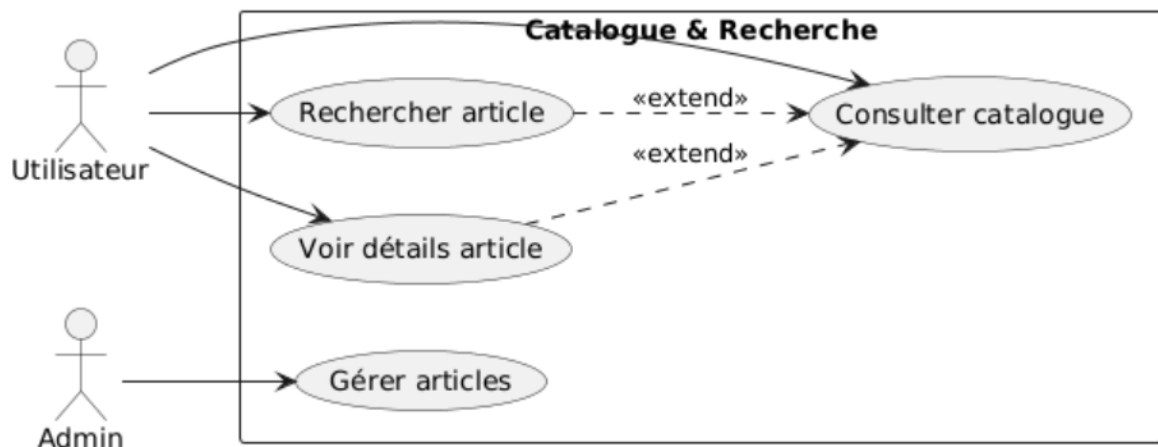
2.1.2 Module Authentification et Profil

Ce module couvre les opérations liées à la gestion de l'identité utilisateur. L'inscription (accessible sans authentification) et la connexion constituent les points d'entrée vers les fonctionnalités personnalisées. Les opérations nécessitant une session active (profil, commandes, panier) incluent la connexion via une relation «include».



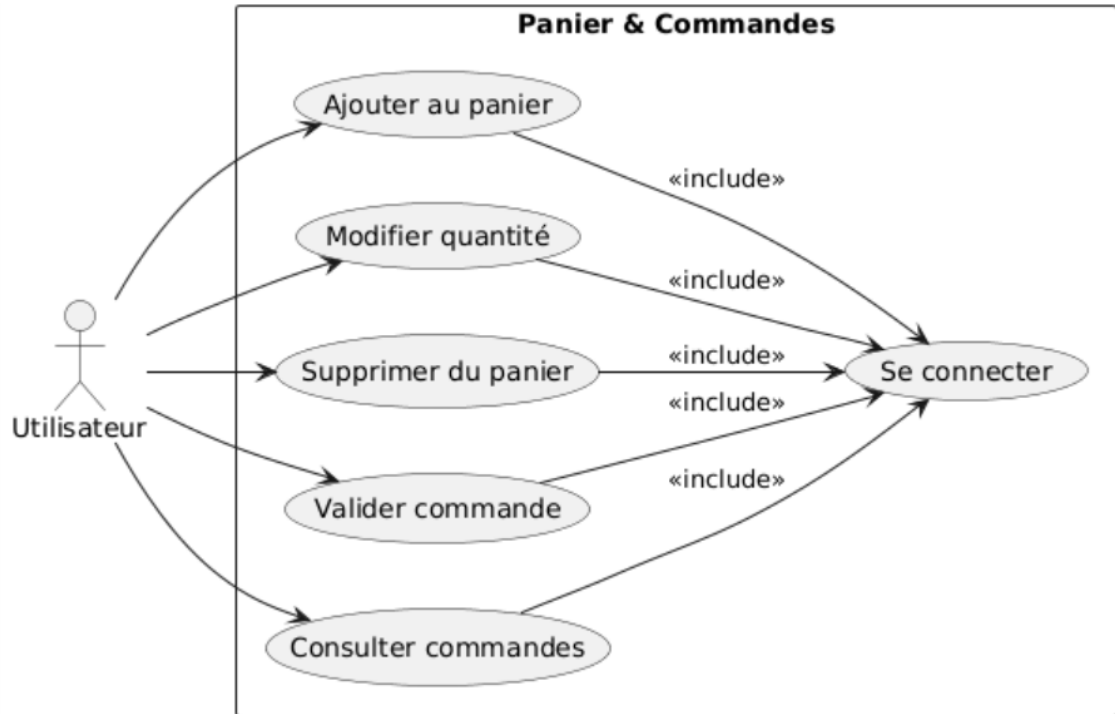
2.1.3 Module Catalogue et Recherche

Ce module permet aux utilisateurs de parcourir le catalogue d'articles, d'effectuer des recherches multicritères (par description, prix, disponibilité), et de consulter les détails d'un article. L'administrateur dispose en plus des fonctionnalités de création, modification et suppression d'articles.



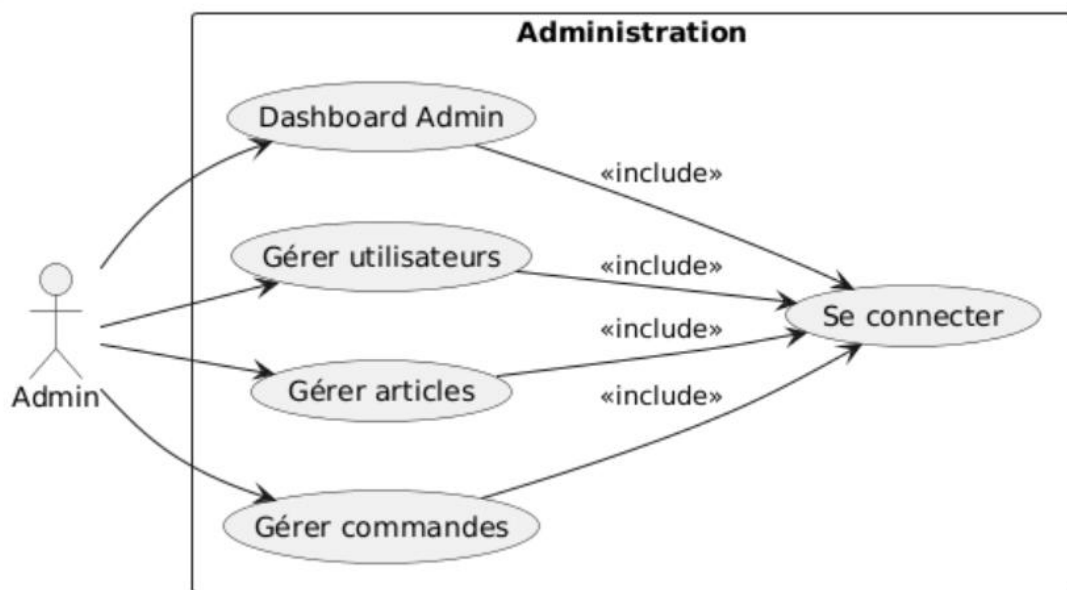
2.1.4 Module Panier et Commandes

L'utilisateur authentifié peut ajouter des articles à son panier, modifier les quantités, supprimer des articles, et valider sa commande. Le module de commandes permet également la consultation de l'historique et le suivi des statuts.



2.1.5 Module Administration

Exclusivement accessible à l'administrateur (ROLE_ADMIN), ce module offre l'accès au tableau de bord, à la gestion des utilisateurs, des articles et des commandes, incluant la modification des statuts.



2.2 Diagrammes de Classes

2.2.1 Entités du Domaine

Le diagramme de classes du domaine métier présente les entités principales et leurs associations :

Le diagramme de classes du système e-commerce représente les principales entités métier ainsi que leurs relations. La classe *User* modélise les utilisateurs de l'application, qui peuvent passer plusieurs commandes et gérer une liste de favoris. La classe *Article* correspond aux produits disponibles dans la boutique, caractérisés par des informations telles que le nom, le prix et la quantité en stock. La classe *Order* représente une commande effectuée par un utilisateur, contenant des informations comme la date, le statut et le prix total. Chaque commande est composée de plusieurs éléments représentés par la classe *OrderItem*, qui permet de gérer la relation entre les commandes et les articles en précisant la quantité et le prix de chaque produit. Enfin, la classe *Favorite* permet de modéliser les articles ajoutés aux favoris par les utilisateurs, établissant ainsi une relation entre *User* et *Article*. Cette modélisation assure une organisation claire des données et facilite le développement et la maintenance de l'application

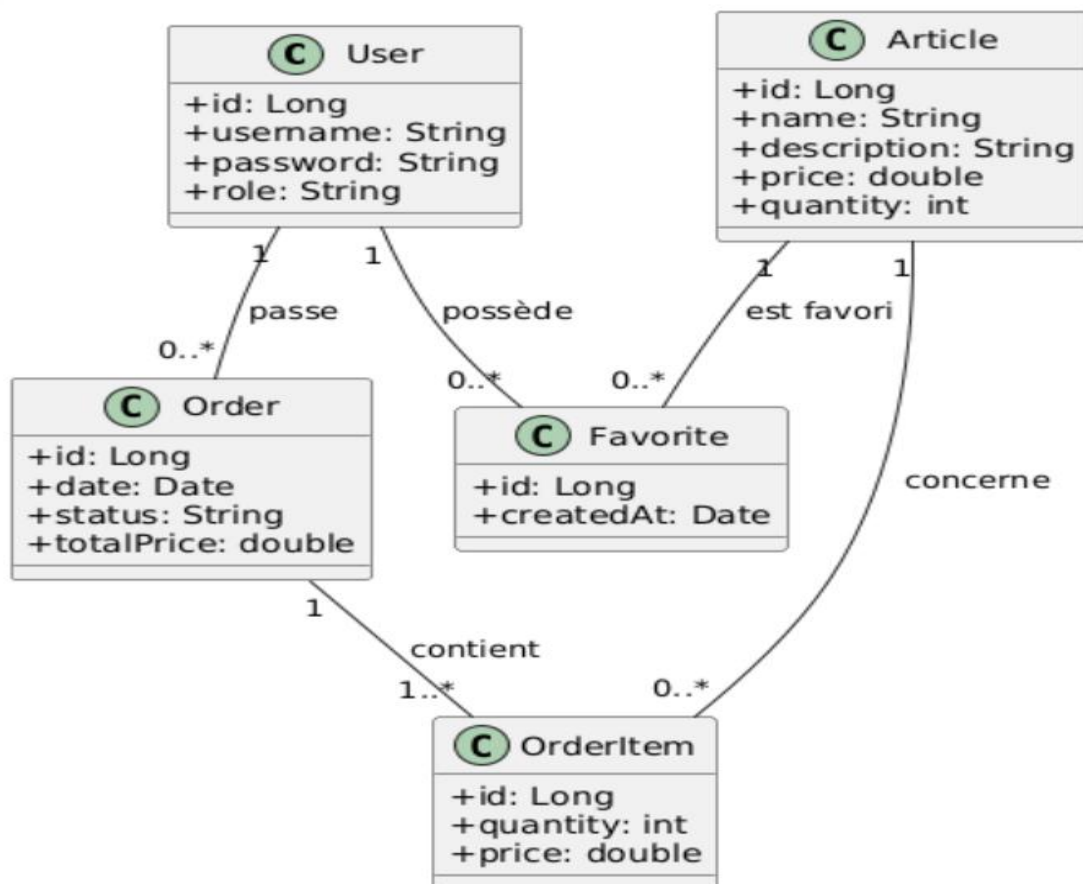


Figure 2.2: Diagramme de classe de l'application ÉléganceShop

Entité	Attributs principaux	Rôle
User	id, username, email, password, role	Compte utilisateur
Article	id, description, prix, dateExpiration, quantiteStock	Produit du catalogue
Commande	id, dateCommande, statut, user, lignes	Commande client
LigneCommande	id, article, quantite, prixUnitaire	Ligne d'une commande
Panier	id, user, lignes	Panier temporaire
LignePanier	id, article, quantite	Ligne du panier

2.2.2 Architecture des Classes Backend

Le backend Spring Boot est organisé selon le modèle **Controllers – Services – Repositories** :

Le backend de l'application Spring Boot est structuré selon une architecture en couches basée sur le modèle **Controllers – Services – Repositories**. La couche **Controller** gère les requêtes HTTP et constitue le point d'entrée de l'application, en recevant les actions des utilisateurs telles que l'authentification, la gestion des articles, des commandes et des favoris. La couche **Service** contient la logique métier de l'application, notamment la validation des données et le traitement des opérations métier. Enfin, la couche **Repository** assure l'accès aux données et l'interaction avec la base de données H2 via Spring Data JPA. Cette architecture favorise la séparation des responsabilités, améliore la maintenabilité du code et facilite l'évolution future de l'application.

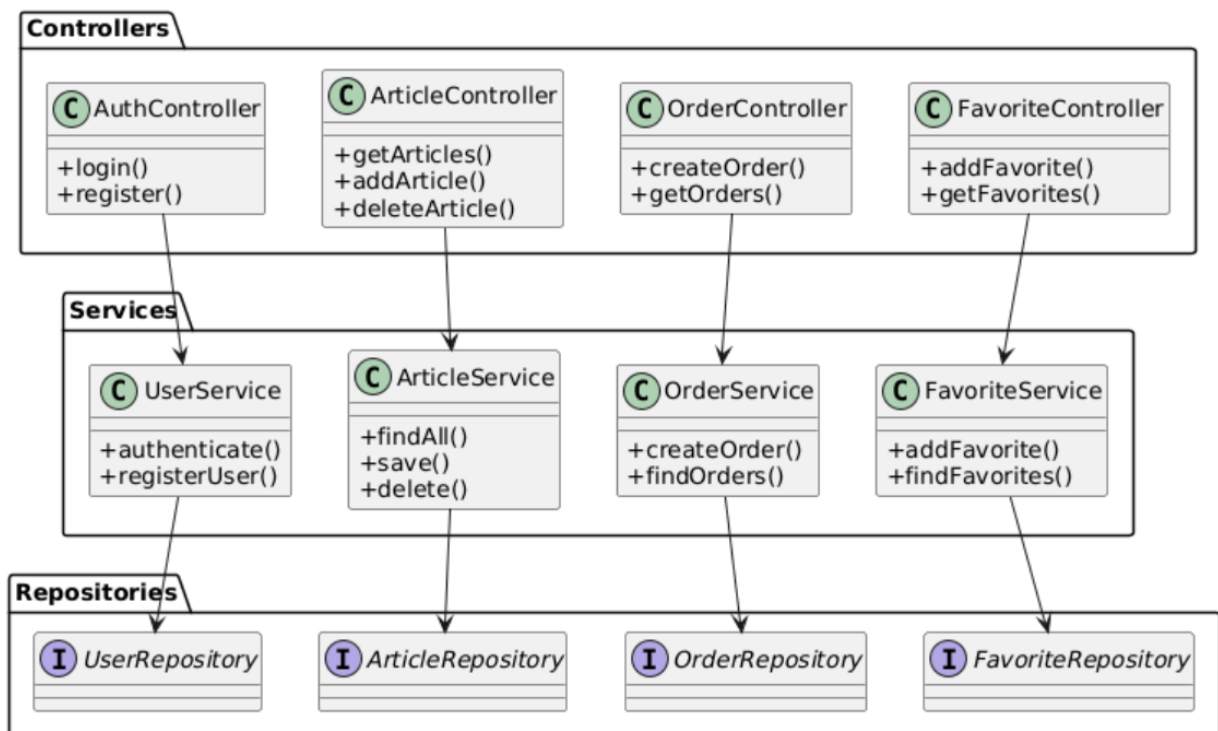


Figure 2.3: Diagramme de classes de l'architecture Backend (Controllers–Services–Repositories)

2.3 Diagrammes de Séquence

Le diagramme de séquence global illustre le fonctionnement général de l'application e-commerce en décrivant les interactions entre l'utilisateur, l'interface utilisateur, les contrôleurs, les services et la base de données. Le processus débute par l'authentification de l'utilisateur via Spring Security, suivie de la consultation des produits à travers des requêtes envoyées au backend. L'utilisateur peut ensuite ajouter des articles au panier et passer une commande, ce qui déclenche l'enregistrement des données dans la base H2. Par ailleurs, le système permet également la gestion des favoris, où les articles sélectionnés sont sauvegardés pour un accès ultérieur. Ce diagramme met en évidence la circulation des données et la séparation des responsabilités entre les différentes couches de l'application.

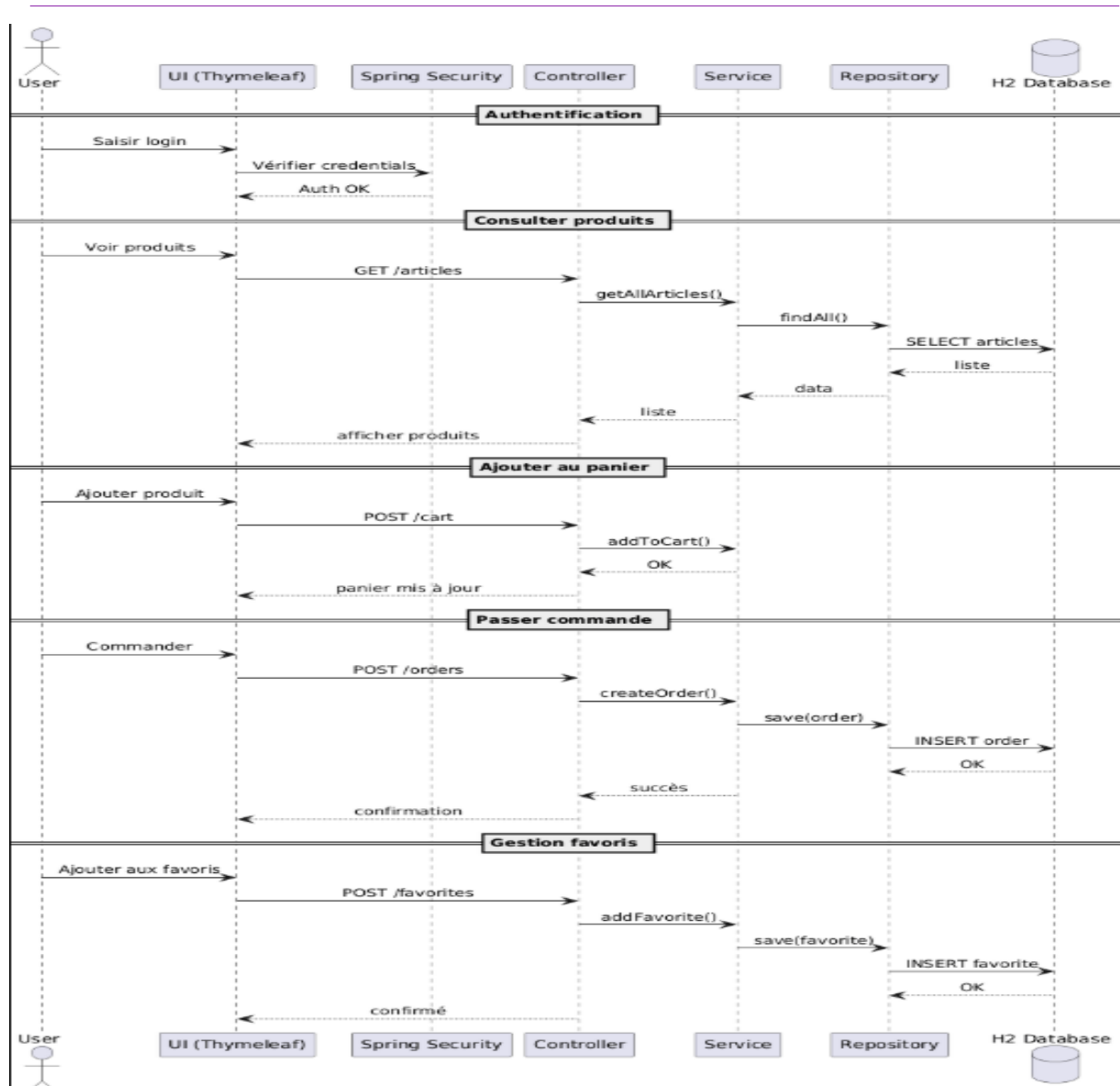


Figure 2.4: Diagramme de séquence global de l'architecture

2.3.1 Authentification d'un Utilisateur

Ce diagramme illustre le processus de connexion : l'utilisateur soumet ses identifiants via le formulaire Thymeleaf, Spring Security intercept la requête POST /login, vérifie

les credentials en base via UserDetailsService, génère un token de session et redirige vers la page d'accueil personnalisée.

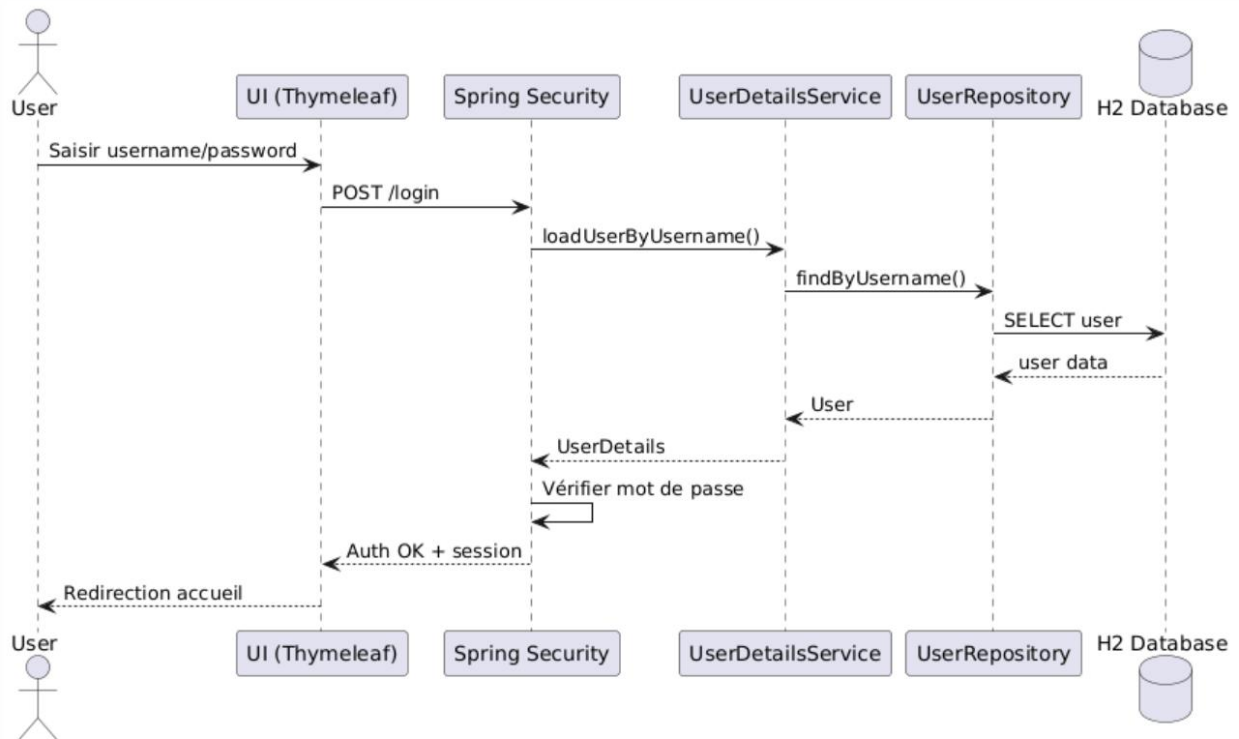


Figure 2.5: Diagramme de séquence authentification d'un utilisateur

2.3.2 Ajout d'un Article au Panier

L'utilisateur clique sur *Ajouter au panier* depuis la boutique. Le PanierController reçoit la requête, délègue au PanierService qui vérifie la disponibilité en stock via ArticleRepository, puis ajoute ou incrémente la ligne dans le panier de l'utilisateur.

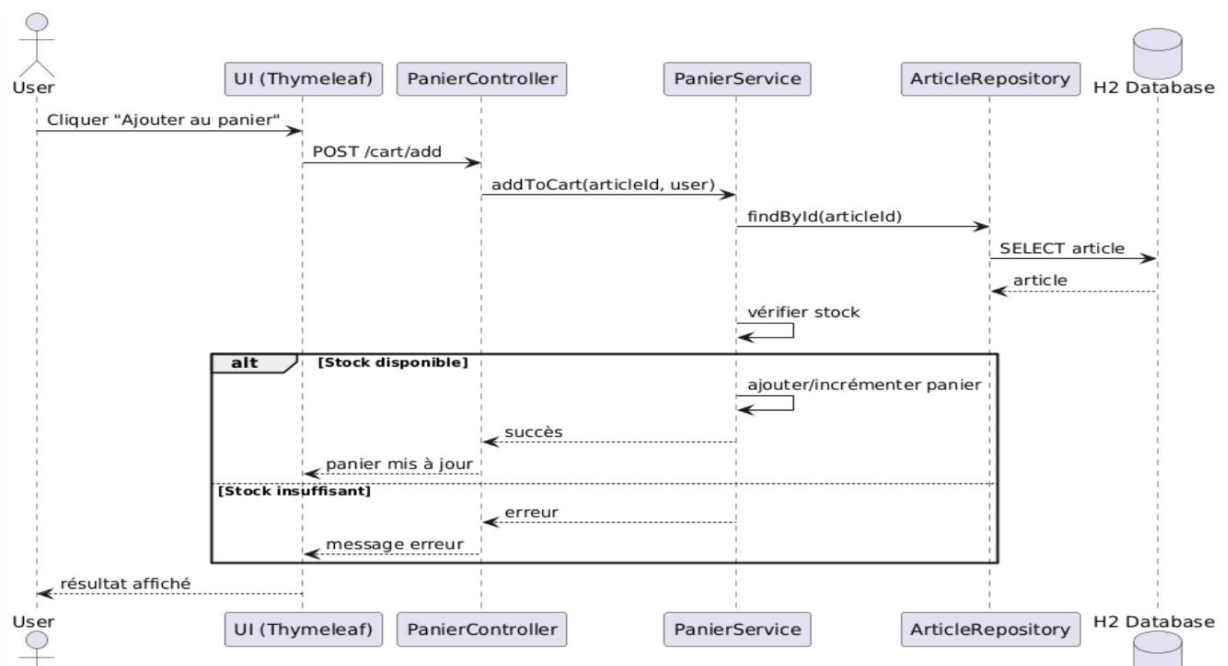


Figure 2.6: Diagramme de séquence ajout au panier

2.3.3 Validation d'une Commande

Depuis la vue panier, l'utilisateur confirme sa commande. Le CommandeService crée une nouvelle Commande, y transfère les LignePanier en LigneCommande, met à jour le stock des articles, vide le panier et persiste la commande avec le statut *EN_ATTENTE*.

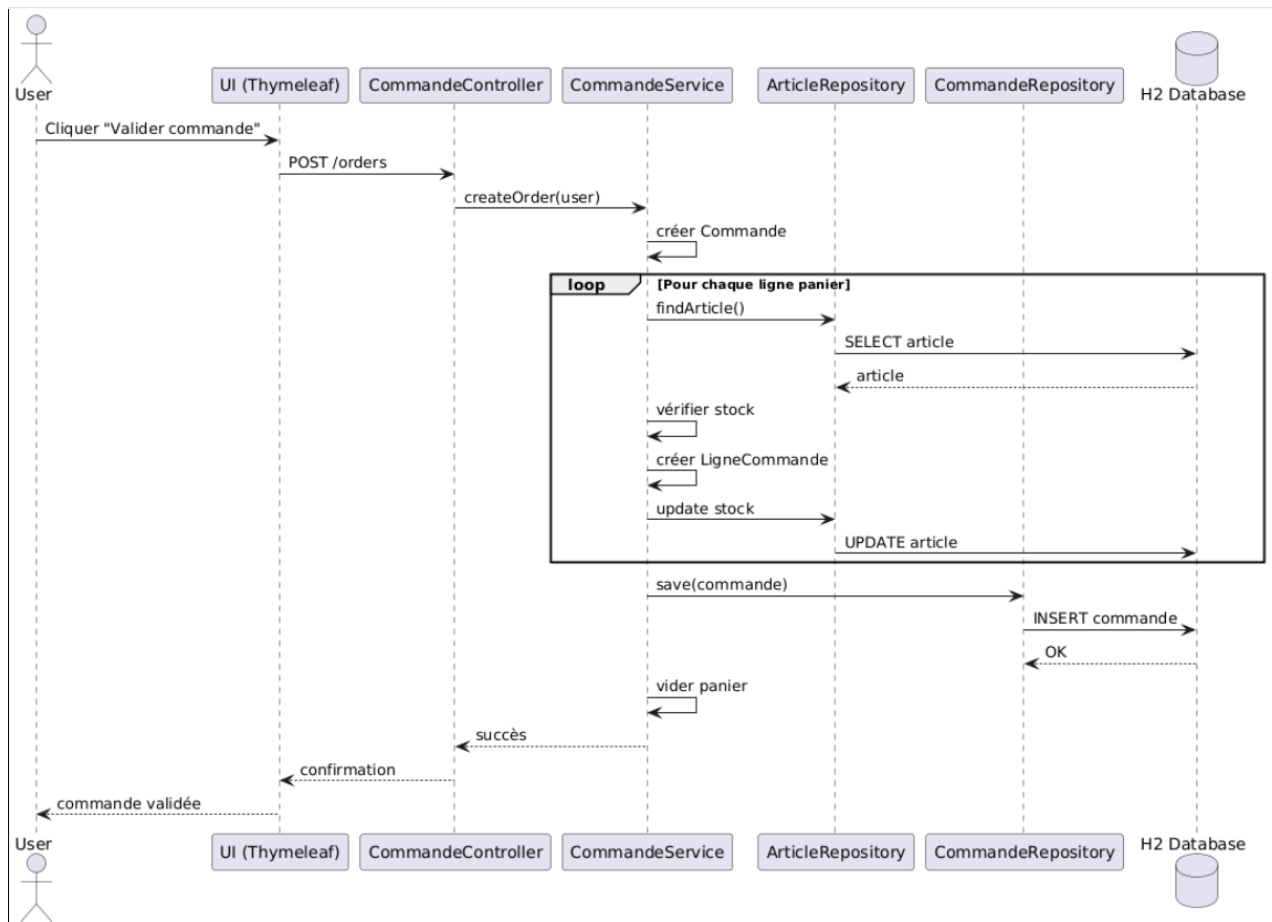


Figure 2.7:Diagramme de séquence validation d'une commande

Chapitre 3

Étude Technique et Implémentation

3.1 Introduction

Ce chapitre présente les choix technologiques retenus pour le développement de l'application ÉleganceShop, ainsi que les détails d'implémentation des principaux composants : configuration Spring Boot, entités JPA, services métiers, sécurité et interfaces utilisateur.

3.2 Technologies Utilisées

3.2.1 Spring Boot



Spring Boot est un framework Java open-source permettant de créer des applications Spring autonomes, prêtes pour la production. Il simplifie la configuration grâce à l'auto-configuration et aux starters, éliminant le besoin de configuration XML volumineuse. Il intègre un serveur Tom-cat embarqué, facilitant le déploiement.

3.2.2 Spring MVC

Spring MVC est le module de Spring gérant le patron Model-View-Controller pour les applications web. Il fournit le dispatcher servlet, les contrôleurs annotés (@Controller, @GetMapping, @PostMapping) et la résolution des vues Thymeleaf.

3.2.3 Spring Security

Spring Security est un framework de sécurité pour les applications Spring, couvrant l'authentification et l'autorisation. Dans ce projet, il gère la connexion des utilisateurs, la protection des routes par rôle, le hachage BCrypt des mots de passe et la protection CSRF.

3.2.4 Spring Data JPA

Spring Data JPA simplifie l'accès aux données en fournissant des interfaces JpaRepository prêtes à l'emploi. Il s'appuie sur Hibernate comme implémentation JPA, permettant le mapping objet-relationnel et la génération automatique des requêtes SQL.

3.2.5 Thymeleaf



Thymeleaf est un moteur de templates Java permettant de créer des vues HTML dy-namiques intégrées nativement avec Spring MVC. Il supporte les expressions EL, les conditions, les boucles et l'intégration Spring Security (attributs sec:authorize).

3.2.6 Bootstrap 5



Bootstrap 5 est un framework CSS/JS front-end assurant la mise en page responsive et les composants UI modernes (navbar, cards, tables, modals, boutons). L'application utilise un thème personnalisé aux couleurs de la marque ÉléganceShop.

3.2.7 Base de Données H2

H2 est une base de données relationnelle embarquée en mémoire, utilisée pour le développement et les tests. Elle offre une console web intégrée accessible à /h2-console et supporte la syntaxe SQL standard. La migration vers MySQL 8 pour la production est transparente grâce à Spring Data JPA.

3.3 Comparaison des Technologies

Critère	Spring Boot	Jakarta EE	Quarkus	Micronaut
Démarrage	Rapide	Moyen	Très rapide	Rapide
Configuration	Auto-config	XML / Annotations	Minimal	Annotations
Écosystème	Très riche	Riche	Croissant	Croissant
Sécurité	Spring Security	JAAS / EE Security	Quarkus Security	Micronaut Sec.
Score (sur 10)	9.2	8.0	8.5	8.0

Critère	Thymeleaf	FreeMarker	JSP
Intégration Spring	Native	Bonne	Bonne
HTML valide	Oui	Non	Non
Sécurité intégrée	Oui (sec:)	Non	Non
Score (sur 10)	9.0	7.5	7.0

3.4 Analyse des Interfaces Utilisateur

3.4.1 Interface Boutique (Côté Client)

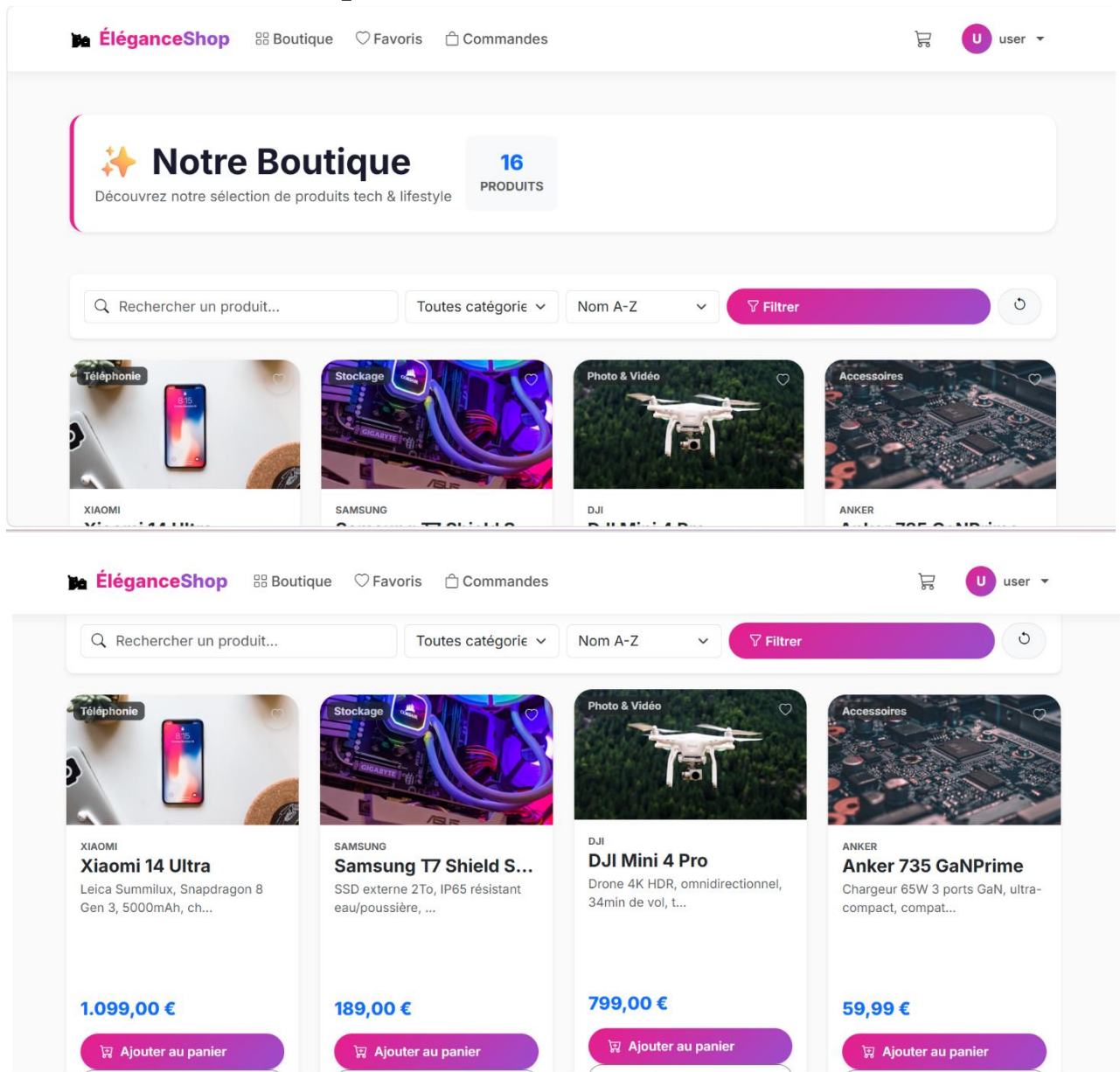


Figure 3.3: Page principale de la boutique ÉléganceShop (vue client authentifié)

L'interface boutique (Figure 3.1) présente les articles disponibles sous forme de cartes visuelles organisées en grille responsive. Chaque carte affiche la catégorie de l'article (badge coloré), l'image, le nom de la marque, la description, le prix et les boutons d'action. La barre de recherche multicritère permet de filtrer les articles par nom, catégorie et ordre de tri.

Pour les utilisateurs authentifiés, le bouton *Ajouter au panier* est actif. Pour les utilisateurs non connectés, le bouton *Se connecter* s'affiche à la place. Le compteur du panier dans la barre de navigation est mis à jour en temps réel.

3.4.2 Tableau de Bord Administrateur

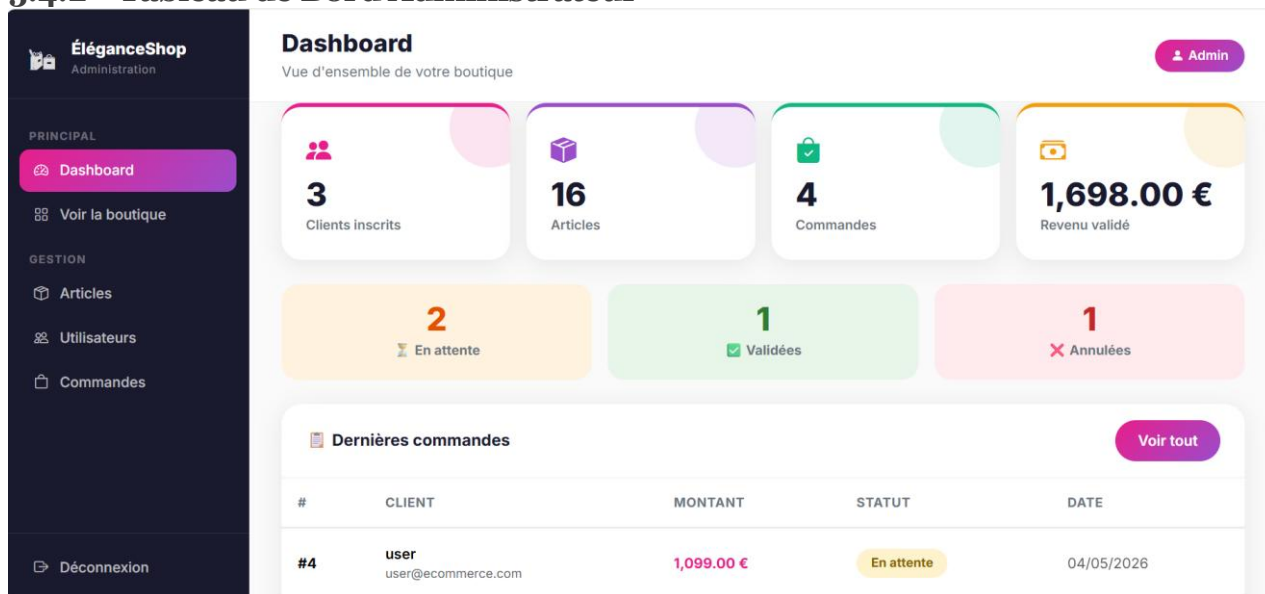


Figure 3.4: Tableau de bord administrateur (/admin/dashboard)

Le tableau de bord administrateur (Figure 3.2) offre une vue synthétique de l'activité de la boutique. Il affiche quatre indicateurs clés : le nombre de clients inscrits, le nombre d'articles disponibles, le nombre total de commandes et le revenu validé en euros. Des cartes de statut indiquent le nombre de commandes en attente, validées et annulées. Un tableau des dernières commandes permet un suivi rapide de l'activité récente. Des liens rapides permettent d'accéder directement aux fonctions d'ajout d'articles, de gestion des utilisateurs et des commandes.

3.4.3 Interface de Connexion

La page de connexion présente un formulaire simple avec les champs *nom d'utilisateur* et *mot de passe*, protégé par un token CSRF. En cas d'échec d'authentification, un message d'erreur est affiché. Le lien d'inscription est visible pour les nouveaux utilisateurs.

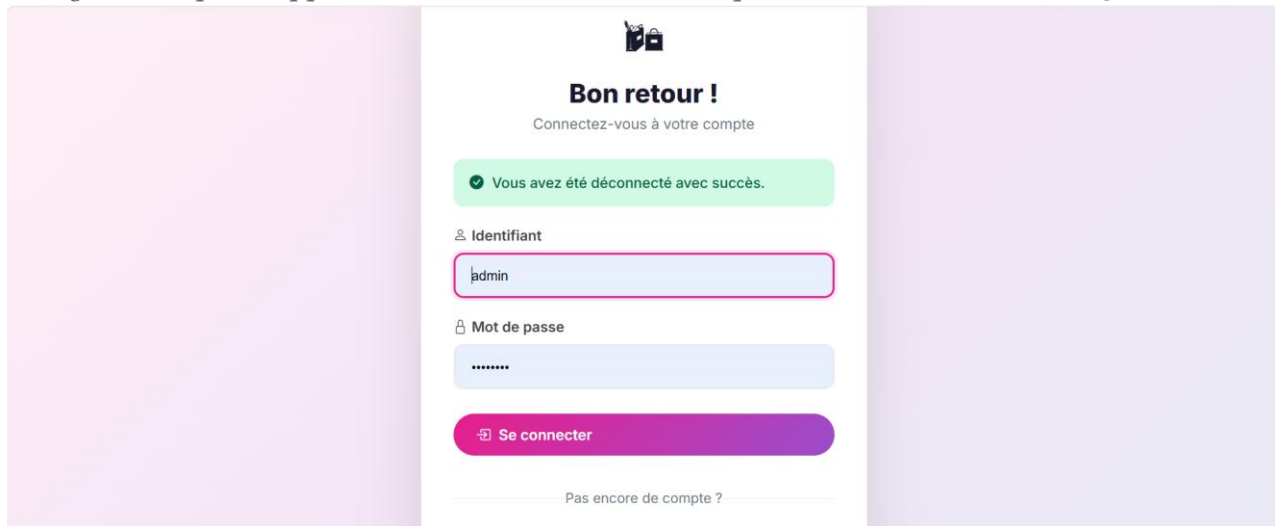


Figure 3.5: Page de login

3.4.4 Gestion des Articles (Admin)

L'interface de gestion des articles permet à l'administrateur de créer un nouvel article via un formulaire validé côté client et côté serveur, de lister tous les articles dans un tableau paginé, de modifier les informations d'un article existant, et de supprimer un article avec confirmation. La recherche multicritère permet de filtrer par description, plage de prix et disponibilité en stock.

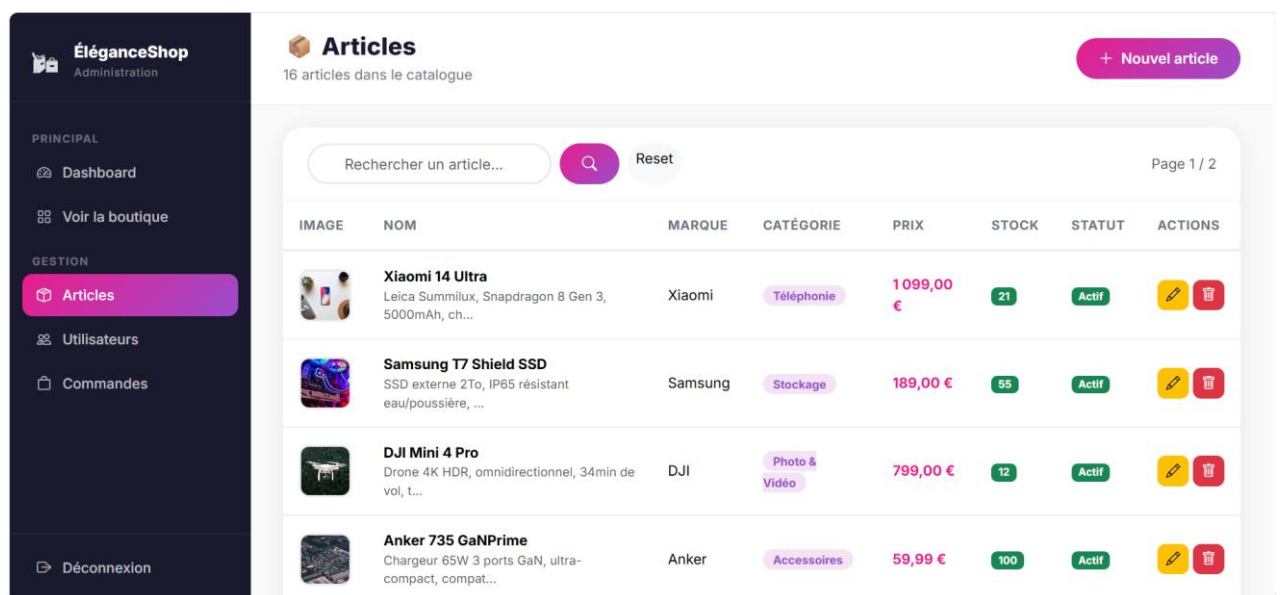


Figure 3.6: Page des gestion des articles

3.4.5 Gestion des Utilisateurs (Admin)

l'interface de gestion des utilisateurs permet à l'administrateur de créer un nouvel utilisateur via un formulaire validé côté client et côté serveur, de lister tous les utilisateurs dans un tableau paginé, de modifier les informations d'un utilisateur existant, et de supprimer un utilisateur avec confirmation. La recherche

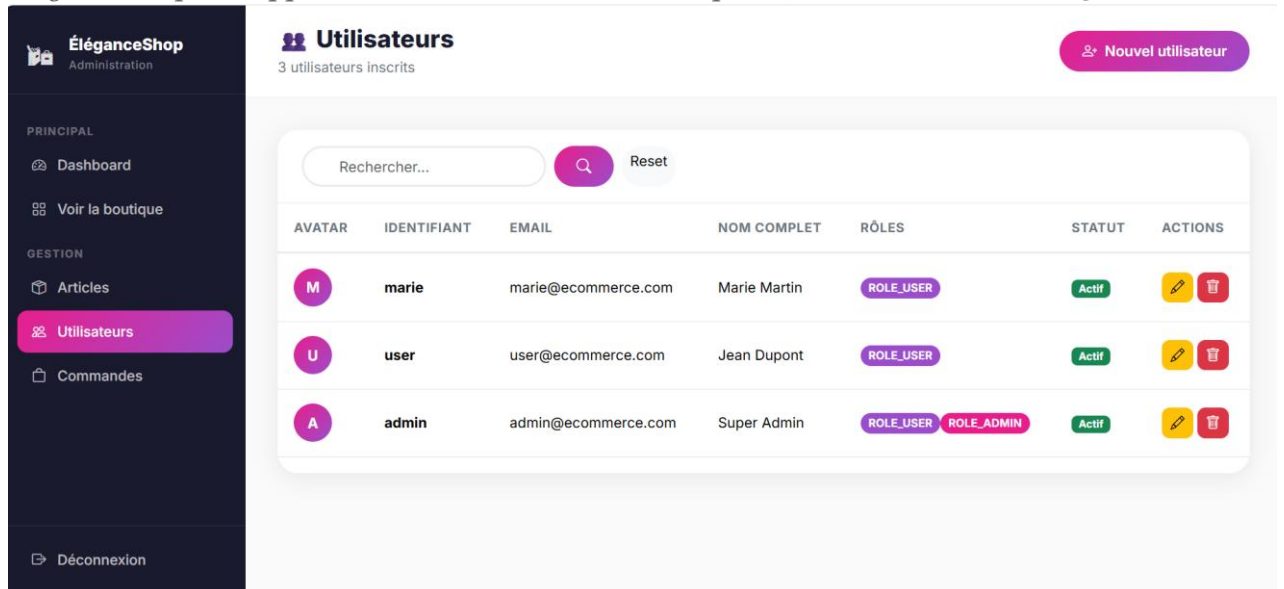


Figure 3.7: Page des gestion des utilisateurs

3.4.6 Gestion des commandes (Admin)

l'interface de gestion des commandes permet à l'administrateur de voir tous les commandes et peut modifier le status (annuler , en attente , valider) et supprimer une commande d'un client



Figure 3.8: Page des gestion des utilisateurs

3.4.7 Gestion du Panier

La vue panier affiche les articles ajoutés avec leurs quantités et prix unitaires. Le total de la commande est calculé dynamiquement. L'utilisateur peut modifier les quantités, retirer des articles et valider la commande. La validation déclenche la création d'une commande et la mise à jour des stocks.

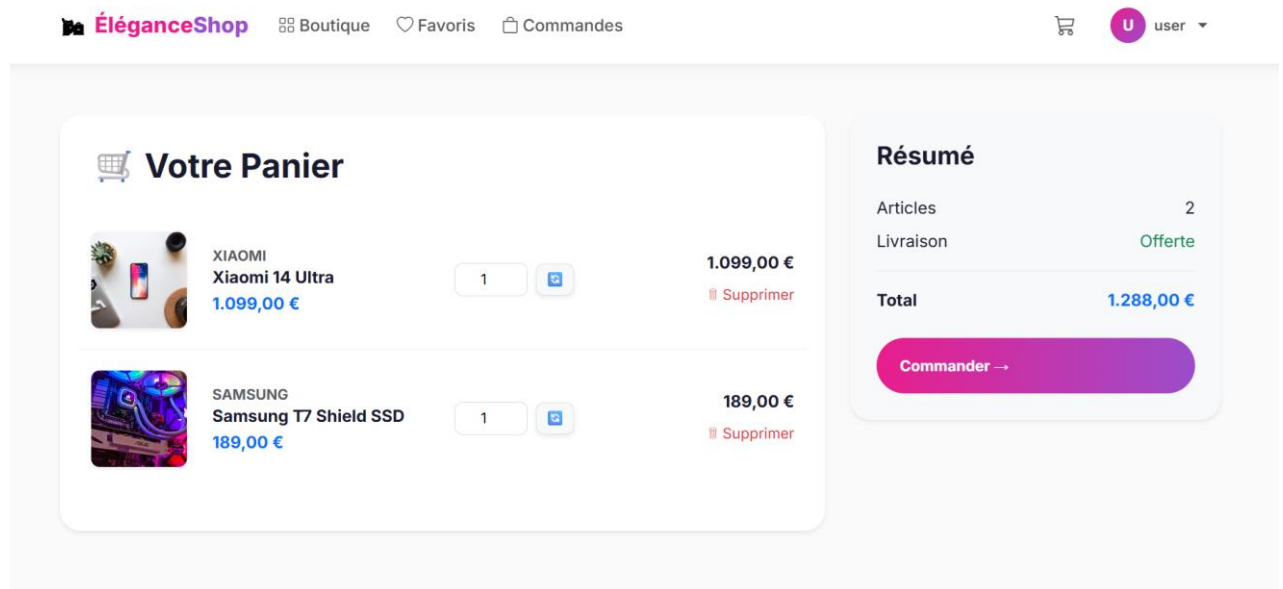


Figure 3.9: Page de gestion du panier

Conclusion Générale

Ce projet a permis de concevoir et de développer une application web e-commerce complète, **ÉléganceShop**, en respectant les exigences du standard Jakarta EE et les principes du développement logiciel moderne. L'ensemble des services demandés a été implémenté avec succès :

- Un **service d'authentification sécurisé** avec Spring Security, BCrypt et gestion des rôles.
- Un **CRUD complet pour les articles** avec recherche multicritère.
- Un **CRUD pour les commandes** avec gestion des lignes et des statuts.
- Un **service de panier** permettant l'ajout, la modification et la validation.
- Des **filtres de sécurité** protégeant les ressources selon le rôle de l'utilisateur.
- Un **tableau de bord administrateur** avec indicateurs clés en temps réel.

L'architecture en trois couches (Présentation / Service / DAO) garantit un faible couplage entre les composants, facilitant la maintenance et l'évolution future de l'application. L'utilisation de Spring Boot, Spring Data JPA, Thymeleaf et Bootstrap constitue un socle technologique robuste et largement éprouvé en milieu industriel.

En perspective, plusieurs améliorations peuvent être envisagées : la migration vers MySQL 8 pour la production, l'ajout d'un système de notifications par email, l'intégration d'un module de paiement en ligne, ou encore le développement d'une API REST pour une application mobile complémentaire.

Ce projet constitue ainsi une base solide pour le développement d'applications web d'entreprise modernes, centrées sur l'expérience utilisateur et conformes aux bonnes pratiques du génie logiciel.

Bibliography

- Spring Team, *Spring Boot Reference Documentation*, Broadcom Inc., 2025. <https://docs.spring.io/spring-boot/>
- Spring Team, *Spring Security Reference*, Broadcom Inc., 2025. <https://docs.spring.io/spring-security/>
- Spring Team, *Spring Data JPA Reference Documentation*, Broadcom Inc., 2025. <https://docs.spring.io/spring-data/jpa/>
- Thymeleaf Team, *Thymeleaf — Natural Templating*, 2025. <https://www.thymeleaf.org/documentation.html>
- The Bootstrap Authors, *Bootstrap 5 Documentation*, 2025. <https://getbootstrap.com/docs/5.3/>
- H2 Database Engine, *H2 Database Documentation*, 2025. <https://h2database.com/html/main.html>
- Oracle Corporation, *MySQL 8.0 Reference Manual*, 2025. <https://dev.mysql.com/doc/refman/8.0/>
- Eclipse Foundation, *Jakarta EE 10 Specification*, 2024. <https://jakarta.ee/specifications/>
- Martin Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2003.
- Craig Walls, *Spring in Action*, 6th edition, Manning Publications, 2022.
- OWASP Foundation, *OWASP Top 10 Web Application Security Risks*, 2024. <https://owasp.org/www-project-top-ten/>